COMP 530 Assignment 7: Optimization

1. The Task

Your task is to implement the exhaustive enumeration algorithm from class, to find the best implementation for an input SQL query.

In `A7.zip`, I've included the A6 skeleton code augmented with some additional code that will be useful for this assignment. Also note that the repository comes with a complete catalog (including statistics which are necessary for this assignment) for the TPC-H benchmark database (the one we used in A6), which is where all of our queries will come from. This catalog is in the file `myCat` in the `bin` directory. To run the system using this catalog, use `scons` to build the SQL parser (option 11) and go into the bin directory and type:

```
>>  sqlUnitTest myCat .
```

This will fire up the database system. You can then enter in any SQL query for parsing and optimization. To change the code so that it prints out the optimized query, go into `main.cc` and un-comment the line that prints the plan. Note that I did not include my own type checking code in `A7.zip`, because all of the queries in the test suite are semantically correct.

2 The Path to Finishing the Assignment

The first thing you'll need to do is to look over the code that I have provided. I have provided some data structures that can hold a relational algebra expression (or equivalently a logical query plan). These data structures are in the file `MyDB_LogicalOps.h` in the new `Execution` source code directory. At the highest level, you need to be able to take a `SFWQuery` data structure and translate it into a tree of `MyDB_LogicalOp` objects with `LogicalTableScan` objects at the leaves, and `LogicalJoin` objects in the internal nodes. This should be done in such a way that (1) the tree of `LogicalOp` objects that you produce correctly implements the input SQL query, and (2) out of all trees of `LogicalOp` objects that are correct, the tree you come up with has the best best (that is, no other tree has a lower cost). The cost is computed as the total number of tuples produced by the plan, summed over all `LogicalOp` objects.

To make this is a bit easier and to ensure that everyone is using exactly the same cost model, I have already implemented code that will allow you to cost `LogicalOp` objects; you can simply use my code without modification to do the costing. This costing functionality is implemented as the `MyDB_Stats` class. This code more or less implements the costing algorithms that we described in class though it does take a few shortcuts to simplify the problem.

I have also included with the distribution a file `Examples.zip` that has a set of text files that contain SQL queries, along with the output of my optimizer for each. This will allow you to test your optimizer. For full credit, you must be able to match or beat the costs that I obtain, when the cost is computed using the `MyDB_Stats` class. Note that you may be graded on some queries that are not distributed with `Examples.zip`.

Here are a few additional things to consider:

(1) Dealing with attribute names in any database system implementation is tricky. The way that we will deal with this is that when we build a logical query plan, we pre-pend the table alias name to all of the attributes in the table. So the query has "`FROM orders AS myOrder`" then the attribute `orders.o_orderkey` becomes `myOrder_o_orderkey`. Every time it is referred to in any schema anywhere in the logical query plan, it should have this name. This ensures that all attribute names are unique and there is never any confusion. This will prevent problems when the same table appears twice in the query plan. This does mean that when you create a `LogicalTableScan` object, you can't just feed it the input table object directly. You will need to pre-pend the allies to each attribute name (fortunately there is an "`alias`" method in the `MyDB_Table` class that does exactly that, returning a copy of the object with all attribute names modified to pre-pend the desired table alias).

(2) At each point in the query plan, the set of attributes that are returned from the `LogicalOp` object must be exactly the set that are needed by any operation above them; no more, and no less.

(3) During the optimization, you will frequently need to check whether expressions in the boolean predicate are being pushed down to the left, to the right, or whether they stay at the top. This will require that you check to see if an attribute or a table are referenced in an expression There are a couple of convenient methods on the `ExprTree` class for exactly this case: `referencesTable` and `referencesAtt`. These will recursively check the entire expression to see if the table or attribute are referenced.

(4) As mentioned above, you will be implementing the recursive enumeration algorithm from class (your implementation will go in `SFWQuery.cc`). What this means is that if you get more than one table, you will consider all possible ways to break them into a left set and a right set, then recursively optimize the left set and the right set. From a practical point of view, these recursive calls will both return optimized `LogicalOp` objects. What you will then do is to look at the statistics returned from these objects, and use the `MyDB_Stats` class to compute the statistics for the join of the two tables that you would get from the two relational algebra expressions represented by those `LogicalOp` objects. Out of all the ways to break the statistics into a left and a right, choose the one with the best cost. Then you create a new `LogicalOp` object corresponding to the resulting join, and return this to the caller. Simple! My `SFWQuery.cc` is only around 300 lines, so there is not a lot of code to implement for this assignment, though it can be a bit intricate.

(5) Finally, execution speed can be a concern with this assignment. The biggest test

query I have given you has seven tables. If your optimizer can come up with a solution for this query in less than 30 seconds, you are OK.

3 Testing, Grading, and Turnin

As usual, zip up your code. If you work with a partner, make sure to submit a readme.

To grade your code, the grader will compile your system on CLEAR using `scons`, and then fire up the database using the catalog `myCat`. The grader will then run a series of queries, checking whether your output looks correct (does the `LogicalOp` tree correctly implement the query?), and whether you obtain a `LogicalOp` tree with the lowest possible cost.

And also: if you work with a partner, **only turn in one submission**. Otherwise, we'll possibly grade your submission twice, and end up getting quite annoyed.